

Excelente. **96.58% de los party-months de 2025 tienen historia completa de 3, 6 y 12 meses**, así que la cobertura es más que suficiente para continuar con el análisis **recent 3m vs prior 9m**.

Hay una precisión: los **party_count** de ambos flags no deben sumarse para obtener personas únicas, porque un mismo **party_id** puede tener meses iniciales incompletos y meses posteriores completos. Lo que nos interesa aquí es que **343,948 de 356,121 observaciones mensuales** tienen historia completa.

Interpretación

PARTY-MONTHS 2025

```
├── 343,948 → 96.58% COMPLETE ✓
└── 12,173 → 3.42% INCOMPLETE
```

Esto nos permite adoptar para el EDA conductual:

`complete_history_flag = 1`

sin perder una parte importante de la población. No eliminaría físicamente los incompletos; simplemente los excluimos cuando una feature necesita baseline de 12 meses.

Ahora sí: construimos **recent 3m** versus **prior 9m**

Yo ajustaría ligeramente el query anterior para que la tabla lleve además controles explícitos de cobertura de ambas ventanas.

```
DROP TABLE IF EXISTS bu_frd.party_trade_features_2025_stage2;
```

```
CREATE TABLE bu_frd.party_trade_features_2025_stage2
USING PARQUET
AS
```

```
WITH base AS (
```

```
    SELECT
        *,
        YEAR(data_date) * 12 + MONTH(data_date) AS month_index
```

```
    FROM bu_frd.party_trade_behavior_monthly_2024_2025_v2
),
```

```
features AS (
```

```
    SELECT
```

*,

```
/* =====  
COVERAGE  
===== */
```

```
COUNT(*) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS observed_months_recent_3m,
```

```
COUNT(*) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING  
) AS observed_months_prior_9m,
```

```
/* =====  
DETERIORATION  
===== */
```

```
SUM(deteriorated_accounts) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS deteriorations_recent_3m,
```

```
SUM(deteriorated_accounts) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING  
) AS deteriorations_prior_9m,
```

```
/* =====  
ACTIVE ADVERSE  
===== */
```

```
SUM(entered_active_adverse_accounts) OVER (  
  PARTITION BY party_id  
  ORDER BY month_index  
  RANGE BETWEEN 2 PRECEDING AND CURRENT ROW  
) AS adverse_recent_3m,
```

```
SUM(entered_active_adverse_accounts) OVER (  
  PARTITION BY party_id
```

```
ORDER BY month_index
RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING
) AS adverse_prior_9m,
```

```
/* =====
PAST DUE
===== */
```

```
AVG(total_past_due_amount) OVER (
PARTITION BY party_id
ORDER BY month_index
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
) AS avg_past_due_recent_3m,
```

```
AVG(total_past_due_amount) OVER (
PARTITION BY party_id
ORDER BY month_index
RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING
) AS avg_past_due_prior_9m,
```

```
/* =====
BALANCE
===== */
```

```
AVG(total_active_balance) OVER (
PARTITION BY party_id
ORDER BY month_index
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
) AS avg_balance_recent_3m,
```

```
AVG(total_active_balance) OVER (
PARTITION BY party_id
ORDER BY month_index
RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING
) AS avg_balance_prior_9m,
```

```
/* =====
DELINQUENCY SHARE
===== */
```

```
AVG(pct_accounts_delinquent) OVER (
PARTITION BY party_id
ORDER BY month_index
RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
) AS avg_pct_delinquent_recent_3m,
```

```

AVG(pct_accounts_delinquent) OVER (
  PARTITION BY party_id
  ORDER BY month_index
  RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING
) AS avg_pct_delinquent_prior_9m,

```

```

/* =====
REVOLVING UTILIZATION
===== */

```

```

AVG(revolving_utilization) OVER (
  PARTITION BY party_id
  ORDER BY month_index
  RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
) AS avg_utilization_recent_3m,

```

```

AVG(revolving_utilization) OVER (
  PARTITION BY party_id
  ORDER BY month_index
  RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING
) AS avg_utilization_prior_9m,

```

```

/* =====
ACTIVE ACCOUNTS
===== */

```

```

AVG(active_accounts) OVER (
  PARTITION BY party_id
  ORDER BY month_index
  RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
) AS avg_active_accounts_recent_3m,

```

```

AVG(active_accounts) OVER (
  PARTITION BY party_id
  ORDER BY month_index
  RANGE BETWEEN 11 PRECEDING AND 3 PRECEDING
) AS avg_active_accounts_prior_9m,

```

```

/* =====
MAX DPD JUMP RECENT
===== */

```

```

MAX(max_dpd_jump) OVER (
  PARTITION BY party_id

```

```

        ORDER BY month_index
        RANGE BETWEEN 2 PRECEDING AND CURRENT ROW
    ) AS max_dpd_jump_recent_3m

FROM base
)

SELECT
    *,

    CASE
        WHEN observed_months_recent_3m = 3
        AND observed_months_prior_9m = 9
        THEN 1 ELSE 0
    END AS complete_12m_baseline_flag,

    /* =====
    CHANGE / ACCELERATION FEATURES
    ===== */

    deteriorations_recent_3m / 3.0
    - deteriorations_prior_9m / 9.0
    AS deterioration_acceleration,

    adverse_recent_3m / 3.0
    - adverse_prior_9m / 9.0
    AS adverse_acceleration,

    avg_past_due_recent_3m
    - avg_past_due_prior_9m
    AS past_due_change_vs_baseline,

    avg_balance_recent_3m
    - avg_balance_prior_9m
    AS balance_change_vs_baseline,

    avg_pct_delinquent_recent_3m
    - avg_pct_delinquent_prior_9m
    AS delinquency_share_change_vs_baseline,

    avg_utilization_recent_3m
    - avg_utilization_prior_9m
    AS utilization_change_vs_baseline,

    avg_active_accounts_recent_3m
    - avg_active_accounts_prior_9m
    AS active_accounts_change_vs_baseline

```

FROM features

WHERE data_date >= DATE('2025-01-31');

¿Por qué dividir deterioros por 3 y por 9?

Porque no sería justo comparar directamente:

3 deterioros en 3 meses

vs

5 deterioros en 9 meses

Los convertimos a intensidad mensual:

Recent = $3 / 3 = 1.00$ por mes

Prior = $5 / 9 = 0.56$ por mes

Acceleration = +0.44

Eso ya representa algo conceptualmente cercano a lo que buscamos para vulnerabilidad.

Primera validación de stage2

Ejecutaría:

```
SELECT
  complete_12m_baseline_flag,
  COUNT(*) AS party_month_count,
  COUNT(DISTINCT party_id) AS party_count,
  ROUND(
    100.0 * COUNT(*) /
    SUM(COUNT(*) OVER ()),
    2
  ) AS pct_party_months
```

FROM bu_frd.party_trade_features_2025_stage2

GROUP BY complete_12m_baseline_flag;

Deberíamos obtener algo muy parecido al **96.58%** anterior.

Después quiero saber qué tan frecuente es el cambio conductual reciente

Este sería el siguiente query importante:

SELECT

COUNT(*) AS party_month_count,

SUM(

CASE

WHEN deteriorations_recent_3m > 0

THEN 1 ELSE 0

END

) AS observations_with_recent_deterioration,

ROUND(

100.0 *

SUM(

CASE

WHEN deteriorations_recent_3m > 0

THEN 1 ELSE 0

END

) / COUNT(*),

2

) AS pct_recent_deterioration,

SUM(

CASE

WHEN deterioration_acceleration > 0

THEN 1 ELSE 0

END

) AS observations_with_deterioration_acceleration,

ROUND(

100.0 *

SUM(

CASE

WHEN deterioration_acceleration > 0

THEN 1 ELSE 0

END

) / COUNT(*),

2

) AS pct_deterioration_acceleration,

SUM(

CASE

WHEN adverse_recent_3m > 0

THEN 1 ELSE 0

END

) AS observations_with_recent_adverse,

```
ROUND(  
  100.0 *  
  SUM(  
    CASE  
      WHEN adverse_recent_3m > 0  
      THEN 1 ELSE 0  
    END  
  ) / COUNT(*),  
  2  
) AS pct_recent_adverse
```

FROM bu_frd.party_trade_features_2025_stage2

WHERE complete_12m_baseline_flag = 1;

Aquí aparecen dos conceptos distintos:

recent deterioration > 0

significa:

“Algo malo ocurrió recientemente.”

Mientras que:

deterioration_acceleration > 0

significa:

“El deterioro reciente es mayor que el comportamiento histórico propio de esa persona.”

El segundo concepto es particularmente interesante para nuestro índice.

También necesitamos percentiles de las variables de cambio

Para las variables monetarias no usaría promedios simples porque ya vimos distribuciones muy sesgadas.

Ejecuta:

SELECT

```
percentile_approx(  
    deterioration_acceleration,  
    array(0.50,0.75,0.90,0.95,0.99),  
    10000  
) AS deterioration_acceleration_pct,
```

```
percentile_approx(  
    past_due_change_vs_baseline,  
    array(0.50,0.75,0.90,0.95,0.99),  
    10000  
) AS past_due_change_pct,
```

```
percentile_approx(  
    balance_change_vs_baseline,  
    array(0.50,0.75,0.90,0.95,0.99),  
    10000  
) AS balance_change_pct,
```

```
percentile_approx(  
    utilization_change_vs_baseline,  
    array(0.50,0.75,0.90,0.95,0.99),  
    10000  
) AS utilization_change_pct,
```

```
percentile_approx(  
    delinquency_share_change_vs_baseline,  
    array(0.50,0.75,0.90,0.95,0.99),  
    10000  
) AS delinquency_share_change_pct
```

FROM bu_frd.party_trade_features_2025_stage2

WHERE complete_12m_baseline_flag = 1;

Esto nos dirá dónde empieza realmente un cambio “inusual”.

Por ejemplo, podríamos descubrir:

deterioration acceleration

P50 = 0

P75 = 0

P90 = 0.33

P95 = 0.67

P99 = 2.00

Entonces podríamos empezar a definir experimentalmente:

normal
moderate change
high change
extreme change

No como reglas definitivas del score, sino para EDA.

Y una prueba que considero indispensable: estabilidad mensual

Como vimos un incremento de deterioros en febrero-abril de 2025, quiero verificar que la aceleración tampoco esté dominada por esos meses.

```
SELECT
  data_date,

  COUNT(*) AS party_count,

  ROUND(
    100.0 *
    SUM(
      CASE
        WHEN deterioration_acceleration > 0
        THEN 1 ELSE 0
      END
    ) / COUNT(*),
    2
  ) AS pct_deterioration_acceleration,

  ROUND(
    100.0 *
    SUM(
      CASE
        WHEN adverse_recent_3m > 0
        THEN 1 ELSE 0
      END
    ) / COUNT(*),
    2
  ) AS pct_recent_adverse,

  percentile_approx(
    deterioration_acceleration,
    0.95,
    10000
  ) AS p95_deterioration_acceleration
```

```
FROM bu_frd.party_trade_features_2025_stage2
```

```
WHERE complete_12m_baseline_flag = 1
```

```
GROUP BY data_date
```

```
ORDER BY data_date;
```

Esto nos permitirá saber si la feature es relativamente estable o si tiene mucho **calendar drift**.

Qué haría ahora

Ejecutaría **stage2** y luego estos tres diagnósticos:

1. cobertura **complete_12m_baseline_flag**;
2. frecuencia de deterioro reciente / aceleración / adverse;
3. percentiles de las cinco variables de cambio.

Después me pasas los resultados. Con eso decidiremos cuáles de estas primeras features pasan realmente al **Feature Catalog de trades**.

Y a partir de ahí estaremos listos para hacer algo importante: **cruzar estas features point-in-time con la primera fraud alert y comprobar si realmente se intensifican durante los 1, 3 o 6 meses anteriores al evento.**